MLFF campaign storage and I/O management specification

Status: Accepted current normative contract for the owner-driven storage subsystem. Historical STOR1-STOR5 design notes are archived in `docs/history/mlff/STOR1_STOR5_HISTORICAL_DESIGN.md`, and the retired DATA9B4 storage/restart contract is archived under `docs/history/mlff/retired_specs/`. Neither is current authority.

Purpose

This specification defines how the MLFF campaign subsystem manages persistent storage and I/O. Storage is strictly subordinate to the accepted P1-P7 scientific and currentness owners: it may change representation, retention, caching, deduplication, archival, recovery, admission, and I/O execution mechanics, and it may never change what the campaign means.

1. Non-negotiable protections

1. **Storage is never a second scientific authority.** It does not decide scientific identity, target membership, selected size, cross-validation acceptance, representative checkpoint choice, final publication membership, qualification outcome, locked activation, or a release verdict. It reads what the owners say.
2. **External inputs are indestructible.** User-supplied source datasets, training sets, replay trajectories, foundation models, and true-label reference material outside the campaign workspace are read-only regardless of path, symlink, configuration, or record reference.
3. **A reference is not an authority.** A path appearing in TOML, SQLite, JSON, a manifest, or an archive manifest confers no deletion authority. Every mutation additionally passes physical containment and ownership validation.
4. **Symlink targets are never traversed.** A campaign-owned symlink object may be unlinked; its target is never followed for traversal or deletion.
5. **Ambiguity retains.** Corrupt, unreadable, or unclassifiable owner state retains the affected artifacts until ownership is repaired or they are independently proven disposable.
6. **A report never authorizes a mutation.** Both report modes are read-only and advisory.
7. **Retired authority stays retired.** STOR1-STOR5 tier policy, `workspace/runs` conventions, `active_process.json`, PID/age/pathname rules, the retired `evaluate`/`verify` stages, DATA7/DATA8 stage folklore, and SELECT2 never regain destructive authority.
8. **Only the current invocation authorizes.** Authority to mutate comes from `--apply` on the invocation being run, and the action comes from the subcommand being run. Persisted configuration cannot carry either: an `apply` or `action` key under `[storage]` is rejected outright rather than honored or silently ignored, and no environment variable is read.
9. **Containment is not ownership.** Being beneath an owner's directory does not make a file that owner's. A directory artifact is destructively recursed into only under an explicit closed-subtree contract from the real owner; otherwise only individually certified children participate and every unexpected descendant is retained.
10. **A mount boundary is an ownership boundary.** A filesystem mounted below an authorized root exposes externally owned bytes through a campaign-looking path. Traversal stops there, and if the platform cannot answer the question the artifact is retained.

2. The one consequential flow

Every consequential storage mutation converges on a single path:

```
real P1-P7 owners
 -> owner views
 -> cross-owner inventory snapshot (transitive protection closure)
 -> one canonical resolved storage policy
 -> immutable owner-bound plan
```

```
-> explicit operator authorization
-> owner-local publication barrier + owner/currentness/filesystem revalidation
-> executor
-> durable audit
-> restart-equivalent product
```

There is exactly one destructive implementation. Reporting, planning, and execution are separate, and only execution mutates.

3. Retention is a cross-owner closure

Eligibility is computed over the **transitive dependency closure of every current or restartable owner**, not by asking each artifact's nominal owner in isolation.

Normative consequences:

- the current P7 publication pins the exact P5 published member checkpoint bytes for as long as that publication resolves and authenticates as current, **including after the P7 attempt retention reference has been released**;
- the current P4 terminal/selected authority pins the P3 immutable evidence its canonical loader and reconciliation chain require;
- a truthful `waiting_for_reference` pins every predecessor artifact needed to resume the exact frozen publication and qualification lineage, including the frozen external-reference request;
- protection is monotone: no owner's cache or history classification can override another current owner's requirement;
- historical evidence becomes archive- or reclamation-eligible only after no current or restartable descendant needs its hot representation;
- the closure is derived from current owner records on every invocation. There is no second persistent dependency database;
- the owner graph is checked for integrity before any consequential planning: owner identities must be unique and every declared dependency edge must resolve to a real owner view. An incomplete or contradictory graph refuses consequential planning instead of planning against a closure it cannot compute.

4. Race-safe mutation

A recent snapshot is not an authorization. P5 and P7 both publish an immutable object and then publish the current pointer that makes it current, so there is a legitimate window in which the object exists and nothing references it.

- Each owner exposes a **publication barrier** for one generation. The publisher holds it across object publication and pointer commit; any storage mutation that could touch that generation's evidence acquires the same barrier across revalidation and mutation.
- Which barriers an operation must hold is **derived from the artifacts the plan actually touches**, not from a fixed list: the generations and run roots named by the planned actions determine the seams acquired. All acquisitions follow one order - run-activity leases, then P5 publication barriers, then P7 publication barriers - so P5 execution, P7 publication, and storage share a single cycle-free order.
- Generation supersession is not a liveness proof. A post-selection run that began under an older selected binding may still be executing, so its owner holds a run-activity lease for its whole write lifetime and storage waits for the owner rather than for a pathname to look old.
- The storage-operation lease serializes storage operations against each other. It does **not** serialize storage against P1-P7 writers, and it is never treated as if it did.
- No broad CampaignStore transaction is held across hashing, compression, or recursive scans.
- Where an owner exposes no race-safe reclamation seam, the affected artifact is retained.

- P3′s accepted publication-window evidence-graph fence remains valid substrate and is reused, not replaced.
- P7 durable `objects/` are ordinary protected evidence and are never an orphan-collection target.

5. Actions and tiers

`storage report`

Owner views supply semantics; bounded physical metadata supplies size. The normal report is **bounded**: it performs one `lstat` per owner artifact, never walks a subtree, and never reads or hashes an owner's O(member-count) topology manifest - it validates only the compact completion anchor. Its cost is therefore a function of how many artifacts the owners declare and not of how much bulk the campaign holds. It reports destructive eligibility as *potential*; the exact closed-subtree certification that authorizes a mutation happens at planning time and is the only thing that reads a full topology manifest. That applies to P7 released-attempt scratch as well as to P5 runs: a released attempt appears in the normal report as a retained container that still needs exact certification, so reporting never scales with attempt bulk and never implies that deletion authority has already been established. A directory's aggregate size is therefore reported as unknown rather than guessed, and the field says so (`size_scope = "unknown_without_deep_audit"`).

The report is a **complete census**: every known campaign family is accounted for, and any workspace tree no owner adapter claims is reported as ambiguous and retained rather than being pooled into a generic bucket or silently omitted. It reports owner and artifact class, current/historical/restart/cache/archive state, coverage semantics, potential reclaim per action, unresolved owners, owner-graph integrity failures, and protected inputs. The SHA-256 receipt cache is reported separately from CampaignStore authority.

Observational commands are side-effect-free

`report`, `report --deep`, `--dry-run` planning, `archive list`, and `archive verify` are observational. They do not create the workspace, the campaign state database, generation roots, or the storage control plane; they do not write acceleration receipts; and they do not deposit a report file in the campaign. The payload is returned to the caller and printed. Describing a campaign is never what brings it into existence, and an uninitialized campaign is reported as uninitialized rather than created.

Consequential storage planning is unavailable while any qualification attempt state cannot be authenticated. An active attempt's references can pin exact P5 checkpoints far outside the P7 tree, so an unreadable state is an unknown cross-owner retention edge; reporting names the exact state and reason, and planning refuses rather than guessing which artifacts it would have protected. Repairing the state restores planning.

Observation is an **invocation-scoped capability**, not a property of the first store the command happens to open. It propagates to every nested owner helper and into every worker thread the invocation spawns, so a helper cannot escape it by calling an ordinary default-creating constructor from a worker. It is enforced at the owner boundary as well as declared: an observational campaign-state open uses a genuinely read-only SQLite connection, and a write attempted through it is refused before anything is committed.

Nothing process-global is toggled to achieve this. Receipt lookups are read-only in an observational context and receipt writes are simply skipped, so an observational report running beside a legitimate consequential operation neither writes anything itself nor disables or redirects that operation's own caching.

`storage report --deep`

Explicit exact recursive physical accounting, symlink and ownership inspection, and largest-artifact detail. Read-only.

`storage cleanup --tier safe`

Zero scientific, restart, qualification, locked, and acceleration-cache capability loss. Candidates are only artifacts an owner has positively released: external record payloads no campaign state row references and that are outside

the publication window; attempt-local bulk of a P7 attempt the owner has released, excluding the attempt record itself, whose terminality is monotonic; and abandoned storage-native staging with no open journal. Safe performs no acceleration-cache eviction.

`storage cleanup --tier cache`

Safe plus eviction of cache/index state whose owner can prove exact reconstruction *at that moment* **and** can prove that no live consumer is depending on it right now.

Reconstructibility alone is not sufficient. The normalized frame cache is reported as exactly reconstructible - the DATA2 source catalog resolves, the cache manifest binds that exact catalog digest, and every per-run source identity and control signature matches, so a rebuild costs one source read per run and reproduces the identical authenticated cache - but P1 exposes no consumer/builder liveness seam, so evicting it could race a reader mid-campaign. It is therefore **retained by both tiers** and reported as `cache_reconstructible = true, cache_evictable = false`. Positive eviction is unlocked by adding a real P1 liveness seam, not by relaxing this rule.

The SHA-256 receipt store is likewise accounted as reusable cache and retained by both tiers: it is the acceleration cache the running storage operation is itself writing to. Anything an owner cannot certify is retained, and a `cache` action over a campaign with no certified family is legitimately a no-op.

`storage cleanup` and campaign-state maintenance

Bounding the campaign store's diagnostic events and rewriting its SQLite file are **two separately planned actions**, not one decision and not a free tail call on the end of a cleanup. Maintenance appears in the plan as its own action or it does not happen; a refused or empty cleanup can never piggyback either mutation.

Excess diagnostic events authorize **pruning only**, and the resolved retention bound is executed exactly - there is no hidden execution floor that would make the plan, the policy identity, and the audit record describe a retention that never happened. Pruning is a small owner-local transaction that takes the write lock up front, so it serializes against any other campaign writer rather than assuming one process owns the database. A **rewrite** exists as its own action only when a fresh observation already satisfies the configured reclaimable-byte or reclaimable-fraction predicate. At execution it re-establishes that predicate and its temporary-space admission *inside a cross-process exclusion* that every campaign-state writer participates in - **including writable construction**, whose schema bootstrap is a real write - and it holds that exclusion through the rewrite itself. The exclusion is one gate per database shared by every store instance in the process, its reentrancy belongs to the acquiring thread rather than to an object, and the advisory lock beneath it is what a second process blocks on. That lock file is campaign-store owner infrastructure: it is never a cleanup, archive, or dedup target, because unlinking a held advisory-lock pathname would split the serialization domain between the old inode and a freshly created one. Measuring free space and then queuing for the database would be a race, not a decision: a second process - normally a second CLI invocation, which no in-process mutex can see - can commit in between and consume exactly the space the rewrite was authorized by. The exclusion is released on success, refusal, and failure alike. Free pages that the prune itself created do not widen the prune into a rewrite: that decision belongs to the next fresh maintenance plan, measured on its own evidence.

Results and audit records distinguish `events_pruned` from `vacuum_performed`. A skipped or failed maintenance is reported truthfully and changes nothing about whether the file actions succeeded.

`storage archive`

A reversible authenticated cold representation of owner-declared historical reproducibility bulk. See section 7.

`storage deduplicate`

Owner-certified immutable deduplication. See section 8.

Retired tiers

`recompute` and `compact` are retired consequential-loss tiers. They are rejected by name; intentionally lossy history pruning requires a separate explicit product decision.

5b. Recursive authority: closed subtrees and containers

A directory-level owner view carries one of two explicit coverage semantics, declared by the owner:

- **closed subtree** - the real owner certifies, from its own authenticated record or exclusive-writer contract, that every traversable descendant belongs to that artifact. The certification bounds what may be acted on: a descendant the owner did not record is a contradiction that withholds authority over the whole artifact rather than being absorbed into it. A recorded member that is *absent* is not a contradiction, because content may legitimately have left the tree into a cold archive.
- **container / open subtree** - the directory is owner-known but its descendants require individual owner views. Unknown descendants stay ambiguous and retained, and the container itself is reported but never acted on.

Certification comes from the owner, never from a filename extension, an age, a stage name, or a storage-authored pathname convention. Concretely:

- a post-selection run root is closed only under P5′s own terminal completion proof, because the run directory is delegated to the configured trainer and P5 alone can say what it produced there. That proof is deliberately two records: a **compact completion anchor** that is O(1) to validate and is the publication commit point, and an immutable **topology manifest** naming every node the run produced. Both are versioned and self-authenticating; a tampered, copied, or self-inconsistent proof makes the run non-certifiable rather than widening what it appears to own, and the superseded single-file development format grants no authority at all;
- a released P7 attempt publishes a versioned typed topology proof bound to the exact released state it was published for. The proof is written first and the released state second, so the state is the commit point: a crash in between leaves a proof bound to a state that is not current, which grants nothing. An aborted attempt that legally reopens as active therefore invalidates its own release proof for free. Every top-level node has to be one the proof recorded, of the recorded kind, before it can be exposed as reclaimable at all, and the superseded development record grants no authority;
- the campaign store's externalized record area is closed by exclusive-writer contract, tightened to exact manifest agreement for a sharded record;
- a superseded target-size execution root records no member manifest, so it is honestly a container: storage reports it and never acts on it.

The recorded topology is **typed and covers directories as well as regular files**. A recursive delete makes directory nodes disappear too, so an unexpected *empty* directory that no recorded node mentions would otherwise be swept away by an action nobody authorized to remove it. Kind is part of the proof, not decoration: a recorded regular file replaced by a directory at the same relative name - or the reverse - is an ownership contradiction, and a comparison that had already reduced both sides to path strings could not see it. A symlink or special object is never made owned by appearing at a familiar name.

Every observation on that path is **no-follow**. Nodes are classified with `lstat`, and the owner's own authority records are opened with `O_NOFOLLOW` and confirmed to be regular files by `fstat` on the opened descriptor - not by a separate `lstat` that a rename could invalidate before the read. A symlink substituted at a completion anchor, topology manifest, attempt state, or released-attempt proof therefore grants nothing, and its target's bytes are never consumed as owner proof.

Recursive cleanup, archive collection and hot reclamation, dedup enumeration, and restore planning recurse destructively only through a freshly revalidated closed-subtree contract. An unexpected descendant that appears

between planning and apply reduces or refuses the planned action rather than being deleted with it, and an archive manifest records only owner-certified members.

One narrow exception exists and is named explicitly: a directory the storage subsystem is the *sole writer* of - its own `.mdstats/storage/staging` scratch - is closed by that exclusivity rather than by an enumerated member set, because enumerating a set from the very tree in question would be circular. It never applies to a directory another component writes into.

6. Resolved storage policy

One canonical resolved policy is shared by every CLI, config, and API entry point. It normalizes aliases before hashing, so equivalent spellings produce one identity, and it binds the requested action and tier, storage and scratch safety reserve, cache eviction limits, SQLite compaction thresholds, archive codec/level/expansion bounds, dedup realization and minimum file size, I/O worker limits, deep-audit bounds, lease timeout, and audit retention.

Policy identity is **action-scoped**: it hashes only the fields the invoked action actually consumes, so changing an archive codec cannot invalidate an unapplied cleanup plan and cannot change a cleanup's identity. Every public policy field is consumed by at least one action; a knob no action reads would be a false contract and is rejected at import.

- Changing a material policy value invalidates an unapplied plan for the actions that consume it.
- A storage policy change never invalidates a P1-P7 scientific identity.
- Presentation-only options are deliberately outside the policy identity.
- Free bytes, inode headroom, and observed saturation are **execution observations** recorded on the plan, not policy. A changed disk causes admission revalidation, not scientific invalidation.
- No environment variable may widen deletion or archive authority; the resolver reads none.
- An unrecognized `[storage]` key is rejected rather than ignored.

7. Cold archive v2

Archive replaces hot bytes only when **all** of the following hold:

1. the owning artifact is historical or otherwise explicitly owner-declared cold-replaceable;
2. no current or restartable dependency closure requires its canonical hot representation;
3. no current owner resolver or currentness validator directly requires the canonical hot file;
4. explicit restore is sufficient to regain the promised historical capability;
5. the archive is authenticated and cataloged before any hot deletion.

Archive is **not** a transparent virtual filesystem. No P1-P7 loader is given an implicit "if missing, read the storage archive" fallback by this subsystem.

Selection, representation identity, and the restore plan

`--root` may **narrow** a selection into an eligible owner artifact; it may never **widen** it to an ancestor. A requested root that is an ancestor of an eligible artifact is rejected outright rather than silently reinterpreted, because an ancestor sweeps in siblings no owner released.

An archive's identity binds its **representation**, not only its logical content: the codec, the compression level, and the serialization contract are part of the identity. Re-encoding the same logical members under a different codec produces a distinct retained representation, and a failed re-encode can never invalidate the representation already retained.

A restore is an **exact owner-bound plan** over named members and containers, computed before anything is staged. `--dry-run` computes the same plan and installs nothing.

Restore never mutates a pre-existing container

Restore distinguishes a directory it creates from one that was already there. A directory this restore creates may receive the archived owner-certified mode and metadata. A **pre-existing** directory is never `chmod`-ed, replaced, or otherwise metadata-mutated merely because the archive contains a directory entry with the same path; only its own owner changes it. Its timestamp may move for the ordinary reason that its entries came back.

A pathname is not a directory. The restore plan binds the exact `(device, inode, type)` of every existing parent it will install through, and verifies each one again immediately before creating or installing anything. Replacing a planned parent with a *different* ordinary directory at the same path - same mode, same type, not a symlink - refuses the restore rather than silently redirecting the installation. Parents this restore creates itself are validated by its own creation and postcondition chain instead.

Protected reauthentication

A reclaim or restore plan binds the exact retained representation it intends to consume: the representation identity, the create-once catalog fields, the manifest content digest, and the blob locator, digest, and size. Authenticating that representation while *planning* is not sufficient, because the bytes a reclamation is about to trust could be replaced or truncated afterwards.

So the exact catalog entry, manifest, and blob are re-read and re-authenticated **inside the protected consequential window** - after the storage-operation lease and every owner seam are held, and before any hot member is removed or any member installed. A mismatch removes nothing and installs nothing; the operation is refused and re-planned against current retained authority.

That check is race-closed because every supported product path that creates, replaces, removes, retires, or operationally updates retained archive control state acquires the same storage-operation lease first; read-only list, verify, and report do not. This is not an OS security boundary: a process that deliberately ignores package ownership and rewrites campaign files is treated as corruption and detected at the next protected authentication point.

Locator containment

The archive catalog and its blobs live under one storage-owned authorized root. A manifest carries a canonical identity-owned relative locator, never an arbitrary filesystem path. An absolute locator, `..`, an empty or alias-normalizing component, a symlink escape, or a locator resolving outside the authorized root is rejected. A manifest field never authorizes reading an external file, even when those bytes satisfy the recorded digest. Member path safety is enforced separately and is not replaced by validating the outer locator. Restore from a user-supplied external archive is not a supported feature of this package; adding it would require an explicit trust/import contract.

Bounded verification and extraction

Archive bytes are authenticated-but-untrusted until every check passes. Before and during extraction the subsystem enforces: supported schema and codec only; canonical workspace-relative member paths with no absolute path, `..`, empty component, normalization alias, or post-normalization duplicate; regular files and directories only, rejecting symlink, hard-link, device, FIFO, and socket members; manifest member-count and total-expanded-byte admission; a hard per-member size bound applied *while streaming*; a cumulative extracted-byte bound; a compressed-to-expanded ratio bound sufficient to refuse decompression amplification before extraction; a campaign-owned staging root with no traversal through archive-created symlinks; exact digest and size authentication before install; and no implicit overwrite of conflicting authoritative bytes.

Durable publication ordering

```
write/stage
 -> flush + fsync content
 -> atomic publish
```

```
-> persist the parent directory entry where supported
-> authenticate the published bytes
-> publish the dependent manifest/catalog/terminal receipt
```

Hot deletion follows an authenticated archive and a durable catalog, then a fresh owner/dependency revalidation, then removal of only still-authorized hot members, then a truthful terminal status. Restore is bounded authenticated staging, durable publication into canonical hot paths, final owner/content authentication, and only then a terminal receipt. The subsystem does not claim power-loss durability stronger than the filesystem provides; where a directory fsync is unavailable the content flush and atomic rename still hold.

The catalog is identity-keyed. There is no `latest` authority. Restoring bytes never promotes historical evidence to current.

8. Immutable deduplication

Deduplication is **direct**: byte-identical members share one inode among themselves. There is no persistent content-addressed store, because a second durable copy of campaign bytes would be a second retention authority with its own lifecycle, reference counting, and failure modes. Removing the last alias therefore releases the inode naturally, with nothing left behind to garbage-collect.

Eligibility is `owner-certified immutability + exact content identity + owner-certified metadata compatibility + closed link ownership + filesystem realization support + race-safe replacement`.

The pre-rename hardlink is staged inside the storage subsystem's own `.mdstats/storage/staging`, keyed by operation identity, and never inside the owner's run directory - a hard crash between the link and the rename therefore leaves storage-owned residue that the existing abandoned-staging lifecycle retires, rather than an unrecorded descendant that would permanently block the very reclamation it interrupted. Staging and the destination must share a filesystem; if they do not, the group is refused rather than downgraded to a copy or an unowned temporary. Whether staging is abandoned is established by the storage-operation lease and the restore journals, never by a process id, an age, or a pathname convention, and cleanup only unlinks staged names: writing or changing metadata through a staging hardlink would mutate the canonical inode itself.

Closed link ownership means the canonical member's link count is fully accounted for: either it has exactly one link, or every one of its links is a known member of the group being deduplicated. A file that is already hardlinked to something outside the campaign is never chosen as the shared canonical inode, because relinking to it would silently give an external file authority over campaign bytes.

- File type and required mode, ownership, and other owner-required metadata must match; equal bytes with divergent metadata are never hardlink aliases.
- A deduplicated family must have no accepted in-place content or material-metadata writer. Only superseded-generation roots participate, because every P1-P7 writer writes into the current generation.
- Replacement is a single atomic rename of a fully linked temporary, so an interruption can never leave a temporary path accepted as canonical.
- Cross-device or unsupported filesystems retain duplicate bytes without a correctness failure.
- Mutable SQLite state, active attempt scratch, and any owner-ambiguous file never participate.
- Dedup changes inode and ctime and therefore invalidates stat-keyed receipts. That is a cache miss and a revalidation, never a scientific state change.

9. Storage-native control plane

The subsystem owns durable state of its own under `.mdstats/storage/`: an identity-keyed archive catalog, archive manifests and blobs, restore journals, a bounded execution audit, operation-serialization state, and restore staging.

- The catalog, manifests, blobs, and journals required to locate, authenticate, resume, or restore a retained cold representation are never deleted or archived away by any action, including audit pruning.
- The execution audit is diagnostic evidence with bounded retention. Losing an old record cannot invalidate scientific currentness, and an incomplete operation is never recorded as complete.
- Restore journals are bounded: a nonterminal journal is recovery authority and is never pruned, while terminal journals are diagnostic evidence retained to a configured bound.
- Catalog fields that establish what a retained representation *is* are create-once. A rewrite that would change an immutable field is rejected; only operational fields may be refreshed.
- Operation-serialization state is operational liveness only. A crashed holder's advisory lock is released by the kernel; recovery never infers authority from a PID, hostname, or pathname.
- No control-plane record carries a scientific currentness decision, and none can make a historical owner artifact current. Plans, audits, and manifests carry no secrets or machine credentials.

10. Admission

Storage does not introduce a second, weaker free-space floor. The campaign's accepted `[execution].minimum_free_disk_gib` reserve and the `[storage]` safety reserve compose as the stricter of the two, because a reserve is a floor. Before a material storage operation the subsystem bounds free bytes against that reserve, inode headroom, and the operation's **peak** temporary amplification — a staged archive blob before reclamation, a restore staging tree plus its installed copy, a dedup temporary link, a SQLite `VACUUM` rewriting the state database beside itself. An admission failure refuses the operation before any mutation and leaves the campaign exactly as it was. Storage pressure never changes target membership, precision, epochs, seed or qualification population, timestep, acceptance thresholds, or a locked policy. I/O concurrency is controlled independently of CPU worker count.

11. Interruption and terminality

Every multi-action operation has an explicit terminality contract.

- **Cleanup**: each removal is independently owner-authorized and race-safe. A crash after a subset is acceptable because each completed removal was individually safe. No terminal `complete` audit is published until every action in that execution reaches a verified terminal disposition. A retry re-inventories and re-plans rather than reusing the old remaining set. Rollback of already-safe deletions is not required.
- **Dedup**: each replacement is exact and idempotent; a retry reauthenticates both the canonical member and the content object before linking.
- **Archive**: archive bytes without an authenticated catalog never authorize hot deletion. An authenticated catalog may truthfully coexist with still-hot members after an interruption; the catalog records that state, and a retry reconciles which members remain and removes only those still authorized under a fresh owner-bound plan.
- **Restore**: partial staged or installed state is never accepted as complete. The terminal receipt follows final canonical-byte authentication. A retry is idempotent for already-present identical bytes and fails closed on conflicts.

Every **normally successful** applied consequential path appends exactly one truthful durable audit record, and a read-only path appends none. An interrupted operation is recorded as `partial`, never as `complete`. The stored record states its own successful publication, so durable evidence of a successful operation never contradicts itself.

Publication and bounded retention are one serialized lifecycle under the storage-operation lease. Retention reads the whole stream and replaces it, so an append that slipped in between would otherwise be rewritten away moments after being reported as published. Retention also refuses to rewrite over a truncated or unauthenticated stream - damage is surfaced, not overwritten - and a retention failure is reported separately: it never unpublishes a record that was written, never rolls back the mutation, and is simply retried by a later operation.

The audit is diagnostic evidence, not scientific authority, so its own storage can fail. When durable audit publication fails the mutation is **not** rolled back and no record is fabricated; instead the outcome is explicitly degraded rather than reported as ordinary success. The status itself carries the difference (`complete_unaudited`, `partial_unaudited`, `refused_unaudited`), the result reports the publication failure, and the CLI says so plainly. This is deliberately pessimistic rather than precise: after an arbitrary write or `fsync` failure a complete record may or may not have reached the file, and the subsystem does not promise a proof of absence it cannot have. What it does promise is that a caller is never told an operation was audited when publication reported failure. Retry and reconciliation start from actual current filesystem and owner state; a destructive action is never replayed merely to manufacture a missing audit record.

12. Production qualification disposition

Routine implementation requires bounded functional, restart, and integrity tests plus representative storage/I/O measurement. Real campaign external-DFT qualification, long target-machine GPU production qualification, and environment-specific HPC storage qualification remain deferred and are not claimed by this specification.